

Algoritmos e Programação Avançada

Trabalho Prático 2

Thiago Urzedo da Silva Paulino

Universidade Federal de Ouro Preto
Instituto de Ciências Exatas e Biológicas
Departamento de Computação

7. Marbles

- Dado n bolas de gude, deve-se comprar caixas para armazená-las. As caixas disponíveis são:
 - Tipo 1, com capacidade n_1 e custo c_1
 - Tipo 2, com capacidade n_2 e custo c_2
- O objetivo é descobrir quantas caixas de cada tipo devem ser compradas, a fim de armazenar todas as bolinhas de gude com o menor preço possível;
- O problema pode ser transformado na equação: $n_1 \cdot x + n_2 \cdot y = N$
- O custo total é dado por: $C = c_1 \cdot x + c_2 \cdot y$
- O objetivo é minimizar C .

7. Marbles

- Para que haja uma solução inteira, o número total n precisa ser divisível pelo MDC de n_1 e n_2 , caso o contrário, o problema não tem solução;
- Para isso, foi feita uma implementação recursiva do Algoritmo de Euclides Estendido, que encontra o MDC de a e b , e determina os inteiros x e y que satisfazem a Relação de Bézout:
 $ax + by = \text{MDC}(a, b)$;
- A partir dessa solução, pode-se obter os valores de x e y . Caso ambos sejam maiores ou iguais a 0, tem-se uma solução válida, sendo possível calcular o custo final.

8. Colour Hash

- O puzzle é formado por duas rodas, cada uma contendo 12 peças. Cada peça é representada por um número inteiro de 0 a 10;
- O objetivo é transformar uma configuração inicial qualquer, na configuração final, vista na imagem ao lado;
- A entrada é representada por 24 números, onde os 12 primeiros são a configuração da roda da esquerda e os 12 últimos, a configuração da roda da direita;
- Ao final, o programa deve imprimir a sequência de passos (rotações), representados por números de 1 a 4, ou imprimir uma mensagem de erro, caso não seja possível resolver o problema em até 16 passos.



8. Colour Hash

- Para a resolução do problema, foi utilizado uma solução baseada em BFS (busca em largura);
- O programa se inicia inserindo estado inicial em uma fila. É feito um loop enquanto a fila não estiver vazia. A cada iteração, um estado é removido da fila e quatro novos estados são gerados a partir de suas possíveis rotações das rodas;
- Para cada novo estado gerado, se já foi visitado, é ignorado. Caso contrário, é marcado como visitado e sua sequência de movimento é atualizada. Se o estado for igual ao final esperado, a sua sequência de movimentos é retornada e o programa encerra. Caso contrário o estado é inserido na fila para ser explorado posteriormente;
- Caso não haja soluções encontradas com menos de 16 passos, o programa termina e imprime a mensagem: “NO SOLUTION WAS FOUND IN 16 STEPS”.

9. From Dusk till Dawn

- O problema envolve encontrar a rota mais curta para Vladimir (um vampiro) entre duas cidades, viajando de trem e considerando as seguintes restrições:
 - Ele só pode viajar entre as 18:00 e 6:00 (ou seja, à noite)
 - Ele deve estar em uma estação durante o dia (das 6:00 às 18:00)
 - Ele precisa beber 1 litro de sangue ao meio-dia (12:00) a cada dia que passa durante a viagem
- O objetivo é minimizar a quantidade de sangue necessária, que equivale ao número de dias completos que ele passa viajando (cada dia requer 1 litro). Se ele chegar ao destino antes do próximo meio-dia, não precisará de sangue para aquele dia;
- A entrada do problema são as rotas dos trens disponíveis (origem, destino, hora de partida e tempo gasto na rota) e a cidade de origem e destino de Vladimir.

9. From Dusk till Dawn

- O problema foi modelado em um grafo, onde os vértices são as cidades e as arestas as rotas de trem entre elas, sendo consideradas apenas as rotas noturnas;
- Para se encontrar o menor caminho, foi utilizado o algoritmo de Dijkstra junto a uma fila de prioridade, onde para cada aresta, é calculado o tempo absoluto de partida e chegada, e a quantidade de sangue necessário para a viagem;
- Ao final, o programa deve imprimir a quantidade necessária de sangue que Vladimir precisará durante o percurso, ou imprimir a mensagem de erro, caso não haja nenhuma rota que Vladimir possa tomar.

10. The Grand Dinner

- O problema envolve um jantar para os times participantes de um campeonato de programação, onde deve-se alocar os participantes em mesas, de modo que:
 - Em uma mesa não haja mais de um membro do mesmo time
 - Nenhuma mesa receba mais pessoas que sua capacidade
- Caso seja possível a alocação, o programa deve imprimir o dígito 1, seguido da lista de mesas escolhidas para os integrantes de cada grupo. Caso o contrário, imprimir o dígito 0;
- Para a resolução, o problema foi modelado como um problema de fluxo em rede, por meio de um grafo bipartido, considerando os conjuntos de times e mesas, utilizando uma matriz de adjacências;

10. The Grand Dinner

- Para que a alocação seja possível, deve-se respeitar o Teorema de Hall: para qualquer subconjunto de equipes, o número total de membros não deve exceder a capacidade total das mesas;
- Usando o algoritmo de Edmonds-Karp, é calculado o fluxo máximo para determinar se a alocação dos membros de determinado grupo é possível de ser feita para as mesas disponíveis;
- Se todo o fluxo for enviado (todos os membros alocados corretamente), é impresso para o grupo, as mesas em que cada membro foi alocado. Caso contrário, é impresso o dígito zero.

11. Chopsticks

- Mr. L usa um conjunto de 3 hashis para comer, sendo dois de tamanho semelhante e outro de tamanho maior;
- O problema envolve descobrir quantos conjuntos de 3 hashis é possível formar, a partir de um conjunto de N hashis de tamanhos distintos, para atender $K+8$ convidados em uma festa de aniversário;
- Para um conjunto de 3 hashis A , B e C , deve-se respeitar as seguintes restrições:
 - $A \leq B \leq C$, não importando o tamanho de C , desde que seja o maior
 - $(A - B)^2$, chamado de “badness” (o quão ruim é o conjunto), deve ser o menor possível, pois os dois menores chopsticks devem ter tamanho semelhante
- A entrada do problema fornece uma lista com os hashis, ordenados por tamanho de forma crescente;

11. Chopsticks

- Para resolução do problema, foi utilizada uma abordagem envolvendo programação dinâmica, onde o objetivo é encontrar o menor badness para formar os conjuntos de hashis;
- Para isso, foi criada uma tabela $dp[i][j]$, onde:
 - i representa o número de conjuntos a serem formados
 - j representa o número de hashis disponíveis
- Para cada elemento de $dp[i][j]$, é feita uma escolha de utilizar ou não o hashi no conjunto com base no comprimento dos demais hashis no conjunto. Ao final das iterações, o valor da última posição de $dp[i][j]$ representa o badness mínimo para formar $K+8$ conjuntos usando todos os N hashis disponíveis;